

Sprint 5 Notes: Media, Embedded Content, and Asset Responsibility

Sprint 5 Goal

The goal of Sprint 5 is to learn how to add media to an HTML page correctly.

Media means content such as:

- images
- audio
- video
- SVG graphics
- embedded pages or maps
- external media files

By the end of this sprint, you should be able to:

- use `img` with `src` and `alt`
- explain why `alt` text matters
- choose responsible image sizes and formats
- understand basic image compression
- understand basic image licensing
- create audio players with `audio`, `controls`, and `source`
- create video players with `video`, `controls`, `poster`, `width`, and `source`
- understand why multiple `source` elements are useful
- recognize the purpose of `track` for captions and subtitles
- create simple inline SVG graphics
- embed external content with `iframe`
- explain replaced elements
- debug common beginner media mistakes

1. Where Sprint 5 Fits in Your Progress

Before Sprint 5, you learned the basic structure of HTML.

You should already know how to:

Sprint	What you learned
Sprint 1	Elements, tags, attributes, void elements, <code>id</code> , <code>class</code> , and basic <code>img</code>

Sprint	What you learned
Sprint 2	Full HTML document structure with <code>DOCTYPE</code> , <code>html</code> , <code>head</code> , and <code>body</code>
Sprint 3	Text meaning with elements like <code>em</code> , <code>strong</code> , <code>abbr</code> , <code>time</code> , <code>code</code> , and <code>pre</code>
Sprint 4	Links, paths, navigation, <code>tel:</code> , <code>mailto:</code> , and <code>address</code>

Sprint 5 builds on those ideas.

Simple idea:

Sprints 1-4 taught you how to structure content.
Sprint 5 teaches you how to add media content responsibly.

A beginner media page still needs the full document structure from Sprint 2.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Media Page</title>
</head>
<body>
  <h1>My Media Page</h1>
  <p>This page contains media examples.</p>
</body>
</html>
```

Important rule:

Media elements usually go inside the body because they are visible page content.

2. What Media Means in HTML

Media is content that is not just normal text.

Examples:

```
Image = visual content
Audio = sound content
Video = moving visual and sound content
SVG = scalable graphic content
iframe = embedded external content
```

Simple mental model:

```
Text content = words on the page
Links = movement between pages or resources
Media = images, sound, video, graphics, and embedded content
```

Media can make a page better, but it can also create problems.

Bad media can make a page:

- slow
- inaccessible
- broken
- legally unsafe
- confusing for users

Good media should be:

- clear
- useful
- optimized
- accessible
- properly licensed
- placed in the correct HTML element

Main Sprint 5 idea:

```
Do not just add media.
Add media correctly and responsibly.
```

3. Images with `img`

The `img` element displays an image on a web page.

Basic pattern:

```

```

The `img` element usually needs two important attributes:

Attribute	Meaning
<code>src</code>	Tells the browser where the image file is
<code>alt</code>	Gives a text description of the image

3.1 `img` Is a Void Element

`img` is a void element.

That means:

- it does not wrap text
- it does not contain child elements
- it does not have a closing tag

Correct:

```

```

Incorrect:

```
</img>
```

Also incorrect:

```
<img>Dog picture</img>
```

Why this is wrong:

```
img cannot contain text.  
img cannot contain child elements.  
img should not have a closing tag.
```

4. The `src` Attribute

`src` means source.

It tells the browser where to find the media file.

Example:

```

```

This means:

```
Go into the images folder.  
Find profile.jpg.  
Display it here.
```

The `src` attribute appears on several media elements, including:

- `img`
- `audio`
- `video`
- `iframe`
- `source`
- `track`

Simple rule:

```
src tells the browser where the file or embedded content comes from.
```

5. The `alt` Attribute

`alt` means alternative text.

It gives a text alternative for an image.

Example:

```

```

The `alt` text is useful because:

- it describes the image if the image does not load
- it helps screen reader users understand the image
- it gives meaning to the image
- it improves the page when images are unavailable

Weak alt text:

```

```

Better alt text:

```

```

Do not write too much.

Describe what matters.

Simple rule:

```
Good alt text explains the useful meaning of the image.
```

6. Media Optimization

Adding media is not only about writing the correct HTML tag.

You also need to think about the actual file.

Media files can be large. Large files can make a page slow.

Sprint 5 focuses on three basic optimization ideas:

Optimization idea	Simple meaning
Size	Do not use an image much larger than needed
Format	Use a suitable file type
Compression	Reduce file weight so the page loads faster

6.1 Image Size

Image size matters.

Bad situation:

The page displays an image at 400px wide.
The actual image file is 5000px wide.

Problem:

The browser downloads a huge file just to show a small image.

Better:

Use an image closer to the size you need.

Simple rule:

Do not serve a huge image when the page only needs a small one.

6.2 Image Format

Different image formats are useful for different cases.

Simple beginner guide:

Format	Common use
JPG / JPEG	Photos
PNG	Images needing transparency or sharp edges
WEBP	Modern compressed images
AVIF	Modern highly compressed images
SVG	Icons, logos, and scalable graphics

Simple rule:

Choose the format based on the type of image and browser support.

6.3 Compression

Compression reduces file size.

Smaller files usually load faster.

There are two broad compression ideas:

Type	Meaning
Lossy compression	Reduces file size by removing some image data
Lossless compression	Reduces file size while preserving original image data more carefully

JPG commonly uses lossy compression.

Simple rule:

Compress media files before using them on a real page.

7. Image Licensing and Asset Responsibility

Images are intellectual property.

That means you cannot assume every image online is free to use.

Unsafe thinking:

I found it on the internet, so I can use it.

Better thinking:

Do I have permission or a valid license to use this asset?

Safe asset sources may include:

- your own image
- an image you purchased
- an image with a license that allows your use
- a Creative Commons image, if you follow the license rules
- a public domain image
- a CC0 image
- a valid fair-use case, if it truly applies

Important:

Always read the license rules before using an image.

Example licensing note:

```
<section>
  <h2>Image License</h2>
  <p>
    The gallery image is used under a Creative Commons license.
    Attribution is required.
  </p>
</section>
```

Simple rule:

Responsible HTML includes responsible asset choices.

8. Audio with `audio`

The `audio` element creates an audio player.

Basic pattern:

```
<audio controls>
  <source src="audio.ogg" type="audio/ogg">
  <source src="audio.wav" type="audio/wav">
  <source src="audio.mp3" type="audio/mpeg">
```

```
    Your browser does not support the audio element.
</audio>
```

Important parts:

Part	Meaning
<code>audio</code>	Creates an audio player
<code>controls</code>	Shows play, pause, and volume controls
<code>source</code>	Gives possible audio files
<code>type</code>	Tells the browser the file format
fallback text	Text shown if the browser cannot use the audio element

8.1 The `controls` Attribute

`controls` is a boolean attribute.

If it is present, the audio controls are turned on.

Correct:

```
<audio controls>
```

Unnecessary:

```
<audio controls="true">
```

The shorter version is the normal HTML style.

Why controls matter:

Without controls, users may not be able to play, pause, or manage the audio easily.

Simple rule:

Audio should usually have controls.

8.2 Audio with `src`

You may see this simple pattern:

```
<audio controls src="song.mp3">
  Your browser does not support the audio element.
</audio>
```

This can work.

But using `source` elements is often better when you want fallback formats.

Better multi-source pattern:

```
<audio controls>
  <source src="song.ogg" type="audio/ogg">
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

8.3 Common Audio Mistake

Weak:

```
<audio src="song.mp3"></audio>
```

Problem:

There are no visible controls.
The user may not be able to play or pause the audio.

Better:

```
<audio controls src="song.mp3">
  Your browser does not support the audio element.
</audio>
```

Best beginner pattern with fallback sources:

```
<audio controls>
  <source src="song.ogg" type="audio/ogg">
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

9. Video with `video`

The `video` element creates a video player.

Basic pattern:

```
<video controls width="400" poster="poster.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Important parts:

Part	Meaning
<code>video</code>	Creates a video player
<code>controls</code>	Shows video controls
<code>width</code>	Sets display width
<code>poster</code>	Shows an image before the video plays
<code>source</code>	Gives possible video files
fallback text	Text shown if the browser cannot use the video element

9.1 Useful Video Attributes

Example:

```
<video controls muted loop width="400" poster="preview.jpg">
```

Common video attributes:

Attribute	Meaning
<code>controls</code>	Shows play, pause, volume, and other controls
<code>muted</code>	Starts the video muted
<code>loop</code>	Repeats the video after it ends
<code>width</code>	Sets the video display width
<code>poster</code>	Shows a preview image before playback

Simple rule:

Use controls when users need to control the video.

9.2 The `poster` Attribute

`poster` gives the image shown before the video plays.

Example:

```
<video controls width="400" poster="images/lesson-preview.jpg">  
  <source src="lesson.mp4" type="video/mp4">  
  Your browser does not support the video tag.  
</video>
```

Simple meaning:

Show lesson-preview.jpg before the video starts.

9.3 Be Careful with Autoplay

Some videos can autoplay, but beginner HTML should be careful with it.

Autoplay can be bad because:

- it may annoy users
- it may use data unexpectedly
- browsers may block or restrict it
- users may need control over media playback

Simple rule:

Prefer user-controlled playback.

10. The `source` Element

The `source` element gives alternative media files.

It is commonly used inside:

- `audio`
- `video`

Example:

```
<source src="video.mp4" type="video/mp4">
<source src="video.webm" type="video/webm">
```

This means:

Browser, try the first source.
If you cannot use it, try the next one.

This is called a fallback pattern.

Why this matters:

Not every browser supports every media format equally.

The `type` attribute helps the browser understand the media format.

Example:

```
<source src="song.mp3" type="audio/mpeg">
```

Simple rule:

```
Use source elements when you want to provide multiple media formats.
```

11. Fallback Text

Fallback text is text shown when the browser cannot use a media element.

Example:

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

The fallback text is:

```
Your browser does not support the audio element.
```

For video:

```
<video controls width="400">
  <source src="video.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

Simple rule:

```
Fallback text gives the user a message if the media element is unsupported.
```

12. The `track` Element

The `track` element adds timed text tracks to media.

It is often used for:

- captions
- subtitles
- chapters
- metadata

Sprint 5 introduces `track`. You will study media accessibility more deeply later.

Basic pattern:

```
<video controls width="400" poster="lesson-poster.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions.vtt" kind="captions" srclang="en" label="English">
  Your browser does not support the video tag.
</video>
```

Important attributes:

Attribute	Meaning
<code>src</code>	Location of the track file
<code>kind</code>	Type of text track
<code>srclang</code>	Language of the track
<code>label</code>	Human-readable name for the track

Example:

```
<track src="captions.vtt" kind="captions" srclang="en" label="English">
```

Simple meaning:

Use the captions.vtt file as English captions.

Important note:

If media contains speech, captions or transcripts are important.

13. SVG: Scalable Vector Graphics

SVG stands for Scalable Vector Graphics.

SVG is different from JPG and PNG.

Simple difference:

JPG / PNG = raster images made of pixels
SVG = vector graphics made of shapes and paths

SVG is useful for:

- icons
- logos
- simple illustrations
- social media icons
- custom bullets
- graphics that need to scale clearly

Why SVG scales well:

SVG is based on shapes and coordinates, not fixed pixels.

That means it can grow or shrink without losing quality in the same way pixel images can.

13.1 Basic Inline SVG

Example:

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">  
  <circle cx="50" cy="50" r="45" stroke="black" stroke-width="4" fill="yellow" />  
</svg>
```

Important parts:

Part	Meaning
<code>svg</code>	The SVG container
<code>width</code>	Display width
<code>height</code>	Display height
<code>viewBox</code>	Internal coordinate system
<code>xmlns</code>	SVG namespace
<code>circle</code>	Draws a circle
<code>path</code>	Draws a custom shape

13.2 SVG `circle`

Example:

```
<circle cx="50" cy="50" r="45" stroke="black" stroke-width="4" fill="yellow" />
```

Meaning:

Attribute	Meaning
<code>cx</code>	Circle center x-position
<code>cy</code>	Circle center y-position
<code>r</code>	Circle radius
<code>stroke</code>	Border color
<code>stroke-width</code>	Border thickness
<code>fill</code>	Inside color

13.3 SVG `path`

A `path` draws a custom shape.

Example:

```
<path d="M35 65 Q50 80 65 65" stroke="black" stroke-width="4"
fill="transparent" />
```

The `d` attribute contains drawing instructions.

Beginner version:

`d` tells the browser how to draw the path.

You do not need to master SVG path syntax yet.

For Sprint 5, recognize that:

`path` is used for custom vector shapes.

13.4 Important SVG Closing Rule

SVG elements like `circle` and `path` should be closed properly.

Good self-closing SVG shape:

```
<circle cx="50" cy="50" r="45" fill="yellow" />
```

Good full SVG example:

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/
2000/svg">
  <circle cx="50" cy="50" r="45" stroke="black" stroke-width="4" fill="yellow" /
>
  <circle cx="35" cy="40" r="5" fill="black" />
  <circle cx="65" cy="40" r="5" fill="black" />
  <path d="M35 65 Q50 80 65 65" stroke="black" stroke-width="4"
fill="transparent" />
</svg>
```

Important correction:

Do not treat SVG circle and path exactly like HTML void elements.
Close them correctly or self-close them.

14. iframe: Embedded Content

The `iframe` element embeds another page inside your page.

It can embed things like:

- maps
- videos
- external pages
- interactive widgets
- small HTML documents

Basic pattern:

```
<iframe
  width="425"
  height="350"
  src="https://www.openstreetmap.org/export/embed.html?bbox=..."
  title="Map of Lagos area, Nigeria">
</iframe>
```

Important attributes:

Attribute	Meaning
<code>src</code>	The page or resource being embedded
<code>width</code>	iframe width
<code>height</code>	iframe height
<code>title</code>	Describes the embedded content
<code>allow</code>	Defines allowed features for embedded content
<code>allowfullscreen</code>	Allows fullscreen mode
<code>referrerpolicy</code>	Controls referrer information
<code>srcdoc</code>	Can contain direct HTML content for the iframe

14.1 iframe Needs a title

Always give an iframe a useful title.

Bad:

```
<iframe src="https://example.com"></iframe>
```

Problem:

Users and assistive technologies may not know what the iframe contains.

Better:

```
<iframe  
  src="https://example.com"  
  title="Example website preview">  
</iframe>
```

Simple rule:

The iframe title should describe the embedded content.

14.2 iframe with Permissions

Some embedded content needs permissions.

Example:

```
<iframe  
  width="400"  
  height="400"  
  src="https://www.youtube.com/embed/example"  
  title="Course video preview"  
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;  
  picture-in-picture; web-share"  
  referrerpolicy="strict-origin-when-cross-origin">
```

```
allowfullscreen>
</iframe>
```

Important parts:

Attribute	Meaning
<code>allow</code>	Lists features the embedded content may use
<code>referrerpolicy</code>	Controls referrer information sent to the embedded page
<code>allowfullscreen</code>	Allows the embedded content to go fullscreen

Beginner rule:

Use iframe attributes intentionally. Do not copy permissions blindly if you do not understand why they are needed.

15. Replaced Elements

A replaced element is an HTML element whose actual content comes from outside the document.

Examples:

- `img`
- `iframe`
- `video`
- `embed`
- `input type="image"`

Simple explanation:

The HTML element marks the place.
The browser brings in the actual content from somewhere else.

Example:

```

```

The HTML does not contain the cat picture itself.

Instead:

The `img` element points to `cat.jpg`.
The browser fetches `cat.jpg`.
The browser displays it in that spot.

Important limit:

CSS can size and position replaced elements,
but it cannot directly edit the external content inside them.

Example:

You can set `iframe` width and height.
But your page CSS cannot freely change the inside of an external website loaded in the `iframe`.

16. Complete Sprint 5 Example Page

Create a file called:

`media-gallery.html`

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Media Gallery</title>
  <meta name="description" content="A beginner HTML media gallery using images,
audio, video, SVG, and iframe.">
</head>
<body>
  <header>
    <h1>Media Gallery</h1>
    <nav>
      <a href="index.html">Home</a>
```

```

    <a href="resources.html">View HTML resources</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <section>
    <h2>Image Example</h2>
    
    <p>This image should be optimized before being used on a real page.</p>
  </section>

  <section>
    <h2>Audio Example</h2>
    <audio controls>
      <source src="audio/sample.ogg" type="audio/ogg">
      <source src="audio/sample.wav" type="audio/wav">
      <source src="audio/sample.mp3" type="audio/mpeg">
      Your browser does not support the audio element.
    </audio>
  </section>

  <section>
    <h2>Video Example</h2>
    <video controls muted width="400" poster="images/video-poster.jpg">
      <source src="video/lesson.mp4" type="video/mp4">
      <source src="video/lesson.webm" type="video/webm">
      <track src="captions/lesson-captions.vtt" kind="captions" srclang="en"
label="English">
      Your browser does not support the video tag.
    </video>
  </section>

  <section>
    <h2>SVG Icon Example</h2>
    <svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://
www.w3.org/2000/svg">
      <circle cx="50" cy="50" r="45" stroke="black" stroke-width="4"
fill="yellow" />
      <circle cx="35" cy="40" r="5" fill="black" />
      <circle cx="65" cy="40" r="5" fill="black" />
      <path d="M35 65 Q50 80 65 65" stroke="black" stroke-width="4"
fill="transparent" />
    </svg>
  </section>

  <section>
    <h2>Embedded Map Example</h2>

```



```

    <iframe
      width="425"
      height="350"
      src="https://www.openstreetmap.org/export/embed.html?bbox=..."
      title="Map showing the course location">
    </iframe>
  </section>

  <section>
    <h2>Asset License Note</h2>
    <p>
      The image used on this page must be owned by me, licensed for use,
      public domain, CC0, or used according to its license terms.
    </p>
  </section>
</main>
</body>
</html>

```

17. Explanation of the Complete Example

Boilerplate

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Media Gallery</title>
</head>
<body>
  ...
</body>
</html>

```

This creates a complete HTML document.

Header and Navigation

```

<header>
  <h1>Media Gallery</h1>
  <nav>

```

```
<a href="index.html">Home</a>
<a href="resources.html">View HTML resources</a>
<a href="contact.html">Contact</a>
</nav>
</header>
```

This gives the page a title and navigation links.

Image Section

```

```

This displays an image.

It includes:

- `src` for the image file
- `alt` for the image description

Audio Section

```
<audio controls>
  <source src="audio/sample.ogg" type="audio/ogg">
  <source src="audio/sample.wav" type="audio/wav">
  <source src="audio/sample.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

This creates an audio player with multiple fallback formats.

Video Section

```
<video controls muted width="400" poster="images/video-poster.jpg">
  <source src="video/lesson.mp4" type="video/mp4">
  <source src="video/lesson.webm" type="video/webm">
  <track src="captions/lesson-captions.vtt" kind="captions" srclang="en"
label="English">
  Your browser does not support the video tag.
</video>
```

This creates a video player.

It includes:

- controls
- muted state
- display width
- poster image
- multiple sources
- captions track
- fallback text

SVG Section

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">  
  <circle cx="50" cy="50" r="45" stroke="black" stroke-width="4" fill="yellow" /  
>  
</svg>
```

This creates an inline vector graphic.

iframe Section

```
<iframe  
  width="425"  
  height="350"  
  src="https://www.openstreetmap.org/export/embed.html?bbox=..."  
  title="Map showing the course location">  
</iframe>
```

This embeds external map content.

It includes a useful `title`.

License Section

```
<section>  
  <h2>Asset License Note</h2>  
  <p>  
    The image used on this page must be owned by me, licensed for use,  
    public domain, CC0, or used according to its license terms.  
  </p>  
</section>
```

This shows responsible asset use.

18. Common Sprint 5 Mistakes and Fixes

Mistake 1: Image Without `alt`

Bad:

```

```

Problem:

The image has no text alternative.

Better:

```

```

Mistake 2: Audio Without Controls

Bad:

```
<audio src="song.mp3"></audio>
```

Problem:

The user may not have controls to play or pause the audio.

Better:

```
<audio controls src="song.mp3">  
  Your browser does not support the audio element.  
</audio>
```

Mistake 3: Video Without Controls

Bad:

```
<video src="movie.mp4" poster="poster.jpg"></video>
```

Problem:

The user may not have controls to play, pause, or manage the video.

Better:

```
<video controls width="400" poster="poster.jpg">  
  <source src="movie.mp4" type="video/mp4">  
  Your browser does not support the video tag.  
</video>
```

Mistake 4: iframe Without Title

Bad:

```
<iframe src="https://example.com"></iframe>
```

Problem:

The embedded content is not described.

Better:

```
<iframe  
  src="https://example.com"  
  title="Example website preview">  
</iframe>
```

Mistake 5: Huge Image File

Bad:

Using a 5000px-wide image when the page only displays it at 400px.

Problem:

The page becomes slower than necessary.

Better:

Resize and compress the image before using it.

Mistake 6: Assuming Online Images Are Free

Bad thinking:

I found the image online, so I can use it.

Better thinking:

I need permission, a license, public domain status, CC0 status, or a valid legal reason.

Mistake 7: Copying SVG Without Closing Elements

Bad:

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">  
  <circle cx="50" cy="50" r="45" fill="yellow">  
</svg>
```

Problem:

The circle element is not closed correctly.

Better:

```
<svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="45" fill="yellow" />
</svg>
```

19. Sprint 5 Debugging Practice

Broken code:

```

<audio src="song.mp3"></audio>
<video src="movie.mp4" poster="poster.jpg"></video>
<iframe src="https://example.com"></iframe>
```

Fixed code:

```


<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>

<video controls width="400" poster="poster.jpg">
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>

<iframe
  src="https://example.com"
  title="Example website preview">
</iframe>
```

Why these fixes are correct:

The image now has alt text.
The audio now has controls.
The video now has controls.
The source elements include type values.
The iframe now has a title.

20. Sprint 5 Practice Task

Create a file named:

course-media-preview.html

Build a course media preview page using only HTML.

Requirements:

1. Use a full Sprint 2 boilerplate.
2. Add one `h1`.
3. Add one image with useful `alt` text.
4. Add one audio player with `controls`.
5. Add at least two audio `source` formats.
6. Add one video with `controls`.
7. Add a `width` attribute to the video.
8. Add a `poster` image to the video.
9. Add at least two video `source` formats.
10. Add one `track` element for captions.
11. Add one inline SVG icon.
12. Add one iframe with a descriptive `title`.
13. Add one licensing note section.

21. Starter Practice Solution

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Course Media Preview</title>
  <meta name="description" content="A beginner HTML page for practicing images,
```



```

audio, video, SVG, iframe, and media responsibility.">
</head>
<body>
  <header>
    <h1>Course Media Preview</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="resources.html">View course resources</a>
      <a href="contact.html">Contact the course team</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Course Image</h2>
      
    </section>

    <section>
      <h2>Audio Preview</h2>
      <audio controls>
        <source src="audio/course-preview.ogg" type="audio/ogg">
        <source src="audio/course-preview.mp3" type="audio/mpeg">
        Your browser does not support the audio element.
      </audio>
    </section>

    <section>
      <h2>Video Preview</h2>
      <video controls width="400" poster="images/course-video-poster.jpg">
        <source src="video/course-preview.mp4" type="video/mp4">
        <source src="video/course-preview.webm" type="video/webm">
        <track src="captions/course-preview.vtt" kind="captions" srclang="en"
label="English">
        Your browser does not support the video tag.
      </video>
    </section>

    <section>
      <h2>Course Icon</h2>
      <svg width="100" height="100" viewBox="0 0 100 100" xmlns="http://
www.w3.org/2000/svg">
        <circle cx="50" cy="50" r="45" stroke="black" stroke-width="4"
fill="yellow" />
        <path d="M30 55 L45 70 L75 35" stroke="black" stroke-width="6"
fill="transparent" />
      </svg>

```

```
</section>

<section>
  <h2>Course Location</h2>
  <iframe
    width="425"
    height="350"
    src="https://www.openstreetmap.org/export/embed.html?bbox=..."
    title="Map showing the course location">
  </iframe>
</section>

<section>
  <h2>Asset License Note</h2>
  <p>
    Media files used on this page must be owned by me, licensed for use,
    public domain, CC0, or used according to their license terms.
  </p>
</section>
</main>
</body>
</html>
```

22. Sprint 5 Revision Questions

Use these questions to test yourself.

Question 1

What does `src` do on `img`, `audio`, `video`, `source`, `track`, and `iframe`?

Answer:

`src` gives the location of the file or embedded content.

Question 2

Why does an image need `alt`?

Answer:

alt gives a text alternative if the image does not load and helps users who cannot see the image.

Question 3

What are three media optimization ideas?

Answer:

Size, format, and compression.

Question 4

What is the difference between raster and vector images?

Answer:

Raster images are made of pixels.
Vector images are made of shapes and paths.

Question 5

Why do SVGs scale well?

Answer:

SVGs are based on shapes and coordinates instead of fixed pixels.

Question 6

What does `poster` do on a video?

Answer:

poster shows an image before the video plays.

Question 7

Why should an iframe have a title?

Answer:

The title describes the embedded content for users and assistive technologies.

Question 8

What is a replaced element?

Answer:

A replaced element is an element whose actual content comes from outside the HTML document.

Question 9

Why use multiple `source` elements?

Answer:

They give the browser fallback media formats to try.

Question 10

Why should you care about media licenses?

Answer:

Because images and media are intellectual property. You need permission or a valid license to use them.

23. Sprint 5 Key Rules

Memorize these rules:

- `img` needs `src` and `alt`.
- `img` is a void element.
- Do not close `img` with `</img>`.
- `src` tells the browser where the file is.

- `alt` gives a text alternative for an image.
 - Optimize images before using them.
 - Do not use huge files when small files will work.
 - Choose the right media format.
 - Compress media files when possible.
 - Do not assume online images are free to use.
 - `audio` creates an audio player.
 - `video` creates a video player.
 - Audio and video should usually have `controls`.
 - `controls` is a boolean attribute.
 - `source` gives fallback media files.
 - `type` tells the browser the media format.
 - `poster` gives a video preview image.
 - `track` can add captions or subtitles.
 - SVG is vector-based.
 - SVG is useful for icons and logos.
 - SVG shapes should be closed correctly.
 - `iframe` embeds another page or resource.
 - `iframe` should have a useful `title`.
 - Replaced elements get their content from outside the document.
-

24. Sprint 5 Mini Glossary

Media

Images, audio, video, graphics, and embedded content used on a web page.

`img`

The element used to display an image.

Example:

```

```

`src`

An attribute that gives the location of a file or embedded content.

`alt`

An attribute that gives text alternative for an image.

Optimization

The process of making media files more suitable for the web by adjusting size, format, and compression.

Compression

Reducing file size so media loads faster.

License

Permission rules that explain how an asset may be used.

`audio`

The element used to add an audio player.

`video`

The element used to add a video player.

`controls`

A boolean attribute that shows media controls.

`source`

An element that provides a media file option.

`type`

An attribute that tells the browser the media file type.

`poster`

An attribute that gives the preview image for a video.

`track`

An element that adds timed text such as captions or subtitles.

SVG

Scalable Vector Graphics. A format for graphics based on shapes and paths.

`svg`

The container element for inline SVG graphics.

`circle`

An SVG element that draws a circle.

`path`

An SVG element that draws a custom shape.

`iframe`

An element that embeds another page or resource inside the current page.

Replaced Element

An element whose actual content comes from outside the HTML document.

Examples:

```
img  
iframe  
video  
embed  
input type="image"
```

25. Sprint 5 Final Self-Check

You are ready to move on from Sprint 5 when you can do these without looking at notes:

- Write an `img` element with `src` and useful `alt` text.
- Explain why `img` is a void element.
- Explain what `src` does.
- Explain what `alt` does.
- Explain why image size matters.

- Choose a basic image format for common cases.
- Explain why compression matters.
- Explain why image licensing matters.
- Write an `audio` element with `controls`.
- Write an audio player with multiple `source` elements.
- Write a `video` element with `controls`, `width`, and `poster`.
- Write a video player with multiple `source` elements.
- Recognize the purpose of `track`.
- Write a simple inline SVG.
- Close SVG shapes correctly.
- Write an `iframe` with `src` and `title`.
- Explain what a replaced element is.
- Fix broken Sprint 5 media code.

26. Sprint 5 Completion Standard

Sprint 5 is complete when you can build a simple media page that includes:

- a complete HTML boilerplate
- one useful image with `alt`
- optimized media thinking
- one audio player with controls
- one video player with controls
- fallback `source` elements
- an introductory `track` pattern
- one inline SVG
- one `iframe` with a descriptive title
- a responsible licensing note

Final mental model:

Do not just add media.
Add media that is useful, accessible, optimized, and legally safe.

Main point of Sprint 5:

HTML media is not only about showing files.
It is about connecting the right file, giving users control, describing content,
and using assets responsibly.